

OrbittAPI

Plataforma SaaS de dados satelitais

Documentacao da Global Solution 2026.1

Disciplinas combinadas: Service-Oriented Architecture (SOA) e Domain-Driven Design (DDD)

Tema do semestre: Industria Espacial

ODS alinhados: 13 (Acao contra a mudanca global do clima) e 2 (Fome zero e agricultura sustentavel)

Integrantes

Nome	RM
Giovanne Charelli Zaniboni Silva	556223
Leonardo Pasquini Baldaia	557416
Gustavo Oliveira de Moura	555827
Lynn Bueno Rosa	551102

FIAP - 1S/2026

Sumario

1. Visao geral do projeto
2. Conexao com o tema Industria Espacial e ODS
3. Arquitetura SOA: dois microservicos e um gateway
4. Visao geral de Domain-Driven Design
5. Strategic Design
 - 5.1 Ubiquitous Language (linguagem ubiqua)
 - 5.2 Bounded Context (contexto delimitado)
 - 5.3 Subdomains: Core, Supporting e Generic
 - 5.4 Context Map e padroes de relacionamento
6. Tactical Design (Building Blocks)
 - 6.1 Entity
 - 6.2 Value Object
 - 6.3 Aggregate e Aggregate Root
 - 6.4 Domain Event
 - 6.5 Domain Service
 - 6.6 Application Service / Use Case
 - 6.7 Repository
 - 6.8 Factory
 - 6.9 Module
7. Arquitetura em camadas e Ports and Adapters (Hexagonal)
8. Anti-Corruption Layer aplicada
9. Erros padronizados via RFC 7807
10. Cache estrategico no Redis
11. Seguranca, JWT e fluxo de autenticacao
12. Mapeamento mestre: conceito DDD para arquivo no codigo
13. Cobertura do backlog (User Stories)
14. Como subir o ambiente (Docker)
15. Endpoints REST e exemplos cURL
16. Testes automatizados
17. Diferenciais implementados
18. Definition of Done
19. Conclusao

1. Visao geral do projeto

OrbittAPI e uma plataforma **Software-as-a-Service** de acesso a dados satelitais. Empresas de qualquer setor (agronegocio, seguradoras, construtoras, consultorias ambientais) podem consumir inteligencia espacial atraves de endpoints REST simples, sem precisar de cientistas de dados ou infraestrutura propria.

A plataforma agrega dados de fontes como NASA, ESA e INPE, processa imagens de satellite e entrega metricas prontas para uso via API: indice de vegetacao (NDVI), uso do solo, risco de alagamento, deteccao de desmatamento e expansao urbana. O modelo de negocio e baseado em assinatura mensal (Free, Startup, Business e Enterprise) com cobranca por volume de chamadas de API.

Este repositorio implementa o **backend** da Global Solution, atendendo simultaneamente os requisitos das disciplinas de SOA (Service-Oriented Architecture) e DDD (Domain-Driven Design). A solucao foi construida como um monorepo Maven com dois microservicos de dominio mais um gateway, todos em Java 21 e Spring Boot 3.3, prontos para subir em containers via Docker Compose.

Stack tecnica

- Java 21 com record types, sealed interfaces e pattern matching
- Spring Boot 3.3.5 (Web, Data JPA, Security, Validation, Actuator)
- Spring Cloud Gateway para roteamento e validacao centralizada de JWT
- PostgreSQL 16 (um schema por bounded context: identity_db e satellite_db)
- Redis 7 para cache de consultas satelitais com TTL de 6 horas
- JWT via biblioteca jjwt 0.12 com assinatura HS384
- BCrypt para hash de senhas (cost factor 12)
- Springdoc OpenAPI para Swagger UI por servico
- JUnit 5, Mockito e AssertJ para testes
- Maven multi-modulo com pom parent e tres modulos filhos
- Docker e Docker Compose para empacotamento e orquestracao local

2. Conexão com o tema Indústria Espacial e ODS

A Indústria Espacial vive uma transformação em escala: o que antes era prerrogativa de agências governamentais hoje é operado por uma cadeia de empresas privadas (SpaceX, Rocket Lab, Planet Labs, Capella Space, entre outras). Essa democratização gerou uma **economia espacial** em torno do uso civil de dados orbitais.

OrbittAPI participa dessa economia oferecendo a camada de software que conecta a saída bruta de satélites (TIFFs multispectrais, GeoJSON de telemetria, índices brutos) com aplicações finais que consomem JSON estruturado. Em vez de exigir que um cliente da agricultura entenda Sentinel-2 ou Landsat 9, ele faz uma chamada GET /vegetation?lat=X&lng=Y e recebe um NDVI já calculado e classificado por nível de vegetação.

ODS principal: 13 - Ação contra a mudança global do clima

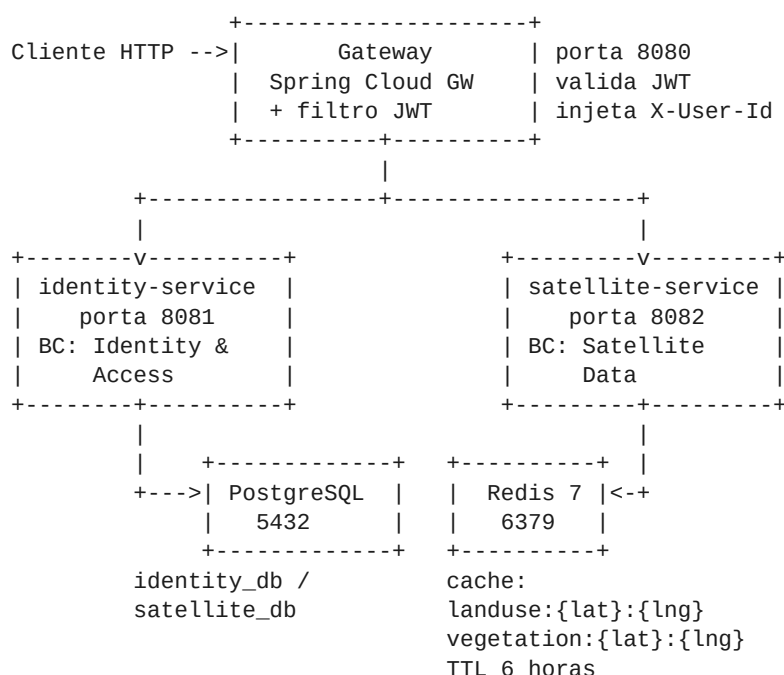
Os índices de vegetação e os mapas de uso do solo permitem monitorar perda de cobertura vegetal, expansão de áreas urbanas sobre matas e impacto de eventos climáticos extremos (alagamentos, secas, queimadas). São insumos diretos para ações de mitigação climática.

ODS secundário: 2 - Fome zero e agricultura sustentável

O NDVI (Normalized Difference Vegetation Index) é um dos principais indicadores para agricultura de precisão. Variações mensais e sazonais no NDVI ajudam produtores a tomar decisões sobre irrigação, plantio, colheita e identificação de pragas. Disponibilizar isso via API barateia o acesso a tecnologia para pequenos produtores.

3. Arquitetura SOA: dois microserviços e um gateway

O backend foi decomposto em três serviços independentes, comunicando-se por HTTP. Cada serviço tem seu próprio Dockerfile, seu próprio schema de banco e pode ser desenvolvido, testado e deployado separadamente. Esta é a definição clássica de uma arquitetura orientada a serviços.



Responsabilidades de cada servico

- **identity-service:** bounded context Identity & Access. Cadastro, login, perfil, geracao/revogacao de API key. Persiste em identity_db.
- **satellite-service:** bounded context Satellite Data. Endpoints /landuse e /vegetation. Persiste auditoria em satellite_db e cacheia respostas no Redis.
- **gateway:** ponto de entrada unico (porta 8080). Roteia HTTP para os servicos, valida JWT em rotas privadas e injeta X-User-Id no request antes de propagar, evitando que cada servico precise revalidar o token.

Por que SOA aqui?

Separar Identity de Satellite Data e mais do que decoracao arquitetural: sao dominios com ritmos de evolucao diferentes (autenticacao e quase estavel, dados satelitais evoluem com novas fontes e novos indices), com cargas diferentes (autenticacao tem picos rapidos, consulta satelital tende a ter consultas mais pesadas), e com times potencialmente diferentes. Manter os dois servicos isolados permite escala-los e evolui-los de forma independente.

4. Visao geral de Domain-Driven Design

Domain-Driven Design (DDD) e uma abordagem para o desenvolvimento de software complexo proposta por **Eric Evans** no livro Domain-Driven Design: Tackling Complexity in the Heart of Software (2003) e expandida por **Vaughn Vernon** em Implementing Domain-Driven Design (2013).

A premissa central e: o software resolve problemas de negocio, e portanto o codigo deve refletir o **modelo do dominio** com a maior fidelidade possivel. DDD propoe um conjunto de praticas para alinhar o modelo mental dos especialistas do negocio com o codigo, dividindo a abordagem em **Strategic Design** (decisoes de larga escala: como dividir o sistema) e **Tactical Design** (blocos de construcao do codigo: entidades, value objects, agregados, eventos).

Este documento percorre cada conceito apresentado em sala e mostra explicitamente como ele aparece no codigo do OrbittAPI.

5. Strategic Design

5.1 Ubiquitous Language (linguagem ubiqua)

A linguagem ubiqua e um vocabulario compartilhado entre desenvolvedores, especialistas de dominio e stakeholders. Todo conceito do negocio deve aparecer no codigo com o mesmo nome usado pelo dominio. Termos genericos como Manager, Helper, Data, Info, Util sao banidos quando ha um termo de negocio disponivel.

Aplicacao no OrbittAPI:

Termo do dominio	Onde aparece no codigo
Account	Account.java (agregado raiz do identity-service)
ApiKey	ApiKey.java (value object)
Email	Email.java (value object com validacao)
Password	Password.java (value object que faz hash no construtor)
SatelliteQuery	SatelliteQuery.java (agregado de auditoria)
Coordinate	Coordinate.java (value object lat/Ing com invariantes)
NdviScore	NdviScore.java (value object com classificacao VegetationHealth)
LandUseDistribution	LandUseDistribution.java (vegetacao, urbano, agua, solo exposto)
VegetationHealth	Enum NONE, SPARSE, MODERATE, DENSE dentro de NdviScore
SatelliteSource	Enum LANDSAT, SENTINEL, MODIS, MOCK

5.2 Bounded Context (contexto delimitado)

Um Bounded Context e uma fronteira explicita dentro da qual um modelo de dominio e consistente. Dois bounded contexts podem ter o mesmo nome para conceitos diferentes (por exemplo, Customer no contexto de Vendas e diferente de Customer no contexto de Cobranca) sem causar ambiguidade, porque cada um vive em sua propria fronteira.

O OrbittAPI tem dois bounded contexts explicitos, refletidos um por microservico:

Bounded Context	Microservico	Linguagem propria
Identity & Access	identity-service	Account, ApiKey, Password, Role, Credentials
Satellite Data	satellite-service	SatelliteQuery, Coordinate, NDVI, LandUse, ImageDate, Source

Nao ha vazamento entre contextos: o satellite-service nao conhece a classe Account; ele recebe um UUID accountId atraves do header X-User-Id injetado pelo gateway. Isso e proposital: cada contexto pode evoluir sem coordenar mudancas com o outro.

5.3 Subdomains: Core, Supporting e Generic

Subdomains classificam partes do negocio de acordo com a sua centralidade estrategica:

- **Core Domain:** o diferencial competitivo da empresa. Merece o melhor time e o modelo mais cuidadoso.
- **Supporting Subdomain:** importante mas nao diferencial. Pode ser construido internamente sem virar o foco principal.
- **Generic Subdomain:** problema resolvido por solucoes de mercado (compra, nao constroi).

Subdomain	Tipo	Justificativa
Satellite Data	Core	E o diferencial competitivo da OrbittAPI. Transformar imagens cruas em metricas REST com
Identity & Access	Supporting	Importante mas nao diferencial. Hoje e construido internamente, mas em uma evolucao po
Billing	Generic	Cobranca por consumo de API e resolvida por solucoes de mercado (Stripe, Iugu). Fora do

5.4 Context Map e padroes de relacionamento

Quando dois ou mais bounded contexts precisam interagir, e necessario definir explicitamente como. Os principais padroes catalogados por Eric Evans sao:

Padrao	Descricao curta
Shared Kernel	Dois contextos compartilham uma porcao pequena de modelo. Mudancas exigem acordo entre os
Customer-Supplier	Um contexto consome o outro. O supplier tem responsabilidade de manter compatibilidade.
Conformist	O downstream se rende ao modelo do upstream sem traduzir, aceitando-o como vem.
Anti-Corruption Layer (ACL)	O downstream coloca uma camada de traducao que isola seu modelo do modelo externo.
Open Host Service	O contexto publica uma API estavel que multiplos consumidores podem usar.
Published Language	Acordo sobre um formato comum (JSON Schema, Avro) usado entre os contextos.
Separate Ways	Decisao explicita de nao integrar. Cada contexto resolve o problema sozinho.
Partnership	Dois times se coordenam ativamente para que ambos os contextos avancem juntos.

Relacoes presentes no OrbittAPI:

- **Identity & Access (upstream) -> Satellite Data (downstream)** via Customer-Supplier + Published Language. O identity emite JWTs com sub=accountId no formato JSON Web Token (linguagem publicada). O satellite consome accountId.
- **Satellite Data -> Fonte externa de dados (NASA/ESA/Mock)** via Anti-Corruption Layer. A porta de dominio SatelliteDataSource isola completamente o modelo do dominio do formato externo. Hoje o adapter e MockSatelliteDataSource; amanha pode ser NasaEarthApiAdapter sem nenhuma mudanca no dominio.

6. Tactical Design (Building Blocks)

6.1 Entity

Uma **Entidade** e um objeto que possui identidade. Dois objetos com os mesmos atributos mas ids diferentes sao considerados **diferentes**. A identidade persiste ao longo do tempo, mesmo quando os atributos mudam.

No OrbittAPI temos duas entidades raizes:

- **Account** (identity-service): identidade UUID; mesmo que email e role mudem, e a mesma conta.
- **SatelliteQuery** (satellite-service): identidade UUID atribuida no momento da execucao.

Trecho: igualdade por identidade no Account

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Account account)) return false;
    return Objects.equals(id, account.id); // <- so o id importa
}

@Override
public int hashCode() {
```



```

        return Objects.hash(id);
    }

```

6.2 Value Object

Um **Value Object** é definido apenas pelos seus atributos. Não tem identidade própria. Dois VOs com o mesmo valor são iguais. São **imutáveis**: qualquer operação retorna uma nova instância. Encapsulam regras de validação no construtor: se você conseguiu construir o objeto, ele é válido. Isso elimina classes inteiras de bugs (coordenada inválida nunca chega no use case).

Exemplo 1: Coordinate

```

public final class Coordinate {

    private final double latitude;
    private final double longitude;

    public Coordinate(double latitude, double longitude) {
        if (latitude < -90.0 || latitude > 90.0) {
            throw new InvalidCoordinateException(
                "Latitude must be between -90 and 90, got " + latitude);
        }
        if (longitude < -180.0 || longitude > 180.0) {
            throw new InvalidCoordinateException(
                "Longitude must be between -180 and 180, got " + longitude);
        }
        this.latitude = latitude;
        this.longitude = longitude;
    }
    // equals/hashCode por valor
}

```

Exemplo 2: Email com normalização no construtor

```

public final class Email {
    private static final Pattern EMAIL_REGEX =
        Pattern.compile("[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$");

    private final String value;

    public Email(String value) {
        if (value == null || value.isBlank())
            throw new InvalidEmailException("Email must not be blank");
        String normalized = value.trim().toLowerCase();
        if (!EMAIL_REGEX.matcher(normalized).matches())
            throw new InvalidEmailException("Email format is invalid: " + value);
        this.value = normalized;
    }
    public String value() { return value; }
}

```

Exemplo 3: Password com hash BCrypt no construtor (factory)

```

public final class Password {
    private static final int MIN_LENGTH = 8;
    private final String hash;

    private Password(String hash) { this.hash = hash; }

    public static Password fromRaw(String raw) {
        validateStrength(raw); // >= 8, 1 número, 1 maiúscula
        return new Password(BCrypt.hashpw(raw, BCrypt.gensalt(12)));
    }
    public static Password fromHash(String hash) { return new Password(hash); }
}

```

```
    public boolean matches(String raw) { return BCrypt.checkpw(raw, hash); }  
}
```

Observacao: a senha em texto plano nunca persiste em lugar nenhum. O construtor publico aceita apenas o hash. A unica forma de criar uma senha a partir de raw e via factory method que ja faz validacao de forza e hash BCrypt.

Exemplo 4: LandUseDistribution com invariante de soma

```
public LandUseDistribution(double vegetationPercent, double urbanPercent,  
                           double waterPercent, double bareSoilPercent) {  
    validateRange("vegetation", vegetationPercent);  
    validateRange("urban", urbanPercent);  
    validateRange("water", waterPercent);  
    validateRange("bareSoil", bareSoilPercent);  
  
    double total = vegetationPercent + urbanPercent + waterPercent + bareSoilPercent;  
    if (Math.abs(total - 100.0) > TOLERANCE) {  
        throw new InvalidLandUseDistributionException(  
            "Land use percentages must sum to 100 (+/- " + TOLERANCE + "), got " + total);  
    }  
    // ...  
}
```

6.3 Aggregate e Aggregate Root

Um **Agregado** é um cluster de objetos (entidades e value objects) que são tratados como uma única unidade transacional. Toda mudança passa pela **raiz do agregado**, que é o ponto de entrada público e o responsável por garantir as **invariantes** do agregado.

Regras de um agregado bem desenhado:

- Apenas a raiz é referenciada externamente. Objetos internos são acessados via a raiz.
- A raiz garante todas as invariantes. Estado inválido nunca chega a ser construído.
- Uma transação modifica um único agregado. Múltiplos agregados se coordenam via eventos.

Agregado Account

Account é a raiz. Email, Password e ApiKey são value objects pertencentes ao agregado. As invariantes protegidas pela raiz incluem:

- Senha sempre persistida como hash BCrypt (nunca em texto plano).
- Email único (validado pelo use case consultando o repositório antes de salvar).
- API key revogada nunca pode ser revogada novamente.
- Eventos de domínio são emitidos como parte da mudança de estado.

```
public class Account {

    private final UUID id;
    private final Email email;
    private Password password;
    private ApiKey apiKey;
    private final AccountRole role;
    private final Instant createdAt;
    private final List<DomainEvent> domainEvents = new ArrayList<>();

    public static Account register(Email email, String rawPassword, AccountRole role) {
        Account account = new Account(
            UUID.randomUUID(),
            email,
            Password.fromRaw(rawPassword),    // <- hash garantido aqui
            ApiKey.generate(),               // <- API key sempre gerada na criação
            role,
            Instant.now()
        );
        account.domainEvents.add(AccountRegistered.now(account.id, account.email));
        return account;
    }

    public void revokeApiKey() {
        if (apiKey.isRevoked()) {
            throw new ApiKeyAlreadyRevokedException(
                "API key for account " + id + " is already revoked");
        }
        this.apiKey = apiKey.revoke();
        domainEvents.add(ApiKeyRevoked.now(id, apiKey.value()));
    }

    public boolean authenticatesWith(String rawPassword) {
        return password.matches(rawPassword);
    }
    // ...
}
```

```
}
```

Agregado SatelliteQuery

Mais simples: agregado de auditoria que registra cada chamada feita aos endpoints de satellite. Quando criado via factory execute(), emite o evento QueryExecuted contendo accountId, type, coordinate e cacheHit (usado para auditoria e para billing futuro).

```
public static SatelliteQuery execute(UUID accountId, QueryType type,
                                     Coordinate coordinate, boolean cacheHit) {
    SatelliteQuery query = new SatelliteQuery(
        UUID.randomUUID(), accountId, type, coordinate, cacheHit, Instant.now()
    );
    query.domainEvents.add(
        QueryExecuted.now(query.id, accountId, type, coordinate, cacheHit));
    return query;
}
```

6.4 Domain Event

Um **Evento de Dominio** representa algo significativo que aconteceu no negocio. E imutavel, tem data/hora e nome no **passado** (Registered, Revoked, Executed). Outros componentes podem reagir a esses eventos de forma desacoplada.

No OrbitAPI, os agregados acumulam eventos durante a execucao de um caso de uso, e o use case publica todos eles apos a persistencia bem-sucedida. A publicacao usa ApplicationEventPublisher do Spring por padrao, mas a interface DomainEventPublisher esta em application/port permitindo trocar facilmente para Kafka, RabbitMQ ou outro mecanismo em uma evolucao futura.

Definicoes dos eventos do projeto

```
public record AccountRegistered(
    UUID accountId, Email email, Instant occurredOn) implements DomainEvent { }

public record ApiKeyRevoked(
    UUID accountId, String apiKeyValue, Instant occurredOn) implements DomainEvent { }

public record QueryExecuted(
    UUID queryId, UUID accountId, QueryType type,
    Coordinate coordinate, boolean cacheHit, Instant occurredOn) implements DomainEvent { }
```

Publicacao dentro do use case (apos persistir)

```
@Transactional
public AuthResponse execute(RegisterAccountCommand command) {
    Email email = new Email(command.email());

    if (accountRepository.existsByEmail(email))
        throw new EmailAlreadyInUseException(email.value());

    Account account = Account.register(email, command.password(), AccountRole.DEVELOPER);
    Account saved = accountRepository.save(account);

    eventPublisher.publishAll(saved.pullDomainEvents()); // <- aqui

    TokenProvider.IssuedToken token = tokenProvider.issue(saved);
    return new AuthResponse(...);
}
```

6.5 Domain Service

Um **Domain Service** é usado quando uma operação do domínio não pertence naturalmente a nenhuma entidade ou value object. Tipicamente envolve mais de um agregado, ou é uma regra de cálculo que não se encaixa como método de um objeto.

No OrbittAPI não identificamos a necessidade de Domain Services neste escopo: as operações do domínio (registrar conta, autenticar, revogar key, consultar landuse, consultar vegetation) couberam todas dentro dos agregados ou dentro de Application Services. Em uma evolução com cálculo de quotas, billing por consumo, detecção de anomalias ou regras de plano (Free/Startup/Business/Enterprise), surgiriam candidatos a Domain Services.

6.6 Application Service / Use Case

Um **Application Service** é a fachada que orquestra o caso de uso: recebe input, carrega agregados via repositórios, chama métodos do domínio para executar a lógica de negócio, persiste as mudanças e publica eventos. **Não contém regra de negócio**: regra fica no agregado. O application service só coordena.

Use Case	Serviço	Responsabilidade
RegisterAccountUseCase	identity	Cria Account, persiste, publica eventos, gera JWT
LoginUseCase	identity	Carrega Account, valida senha, gera JWT
GetMyProfileUseCase	identity	Carrega Account pelo id do JWT, retorna DTO de perfil
RevokeApiKeyUseCase	identity	Carrega Account, executa revokeApiKey(), publica evento
GetLandUseUseCase	satellite	Consulta cache; se miss, chama SatelliteDataSource; persiste auditoria
GetVegetationUseCase	satellite	Mesmo fluxo do landuse mas para NDVI

Exemplo completo: GetLandUseUseCase

```
@Service
public class GetLandUseUseCase {

    private final SatelliteDataSource dataSource;
    private final SatelliteQueryCache cache;
    private final SatelliteQueryRepository queryRepository;
    private final DomainEventPublisher eventPublisher;

    public GetLandUseUseCase(SatelliteDataSource dataSource,
                            SatelliteQueryCache cache,
                            SatelliteQueryRepository queryRepository,
                            DomainEventPublisher eventPublisher) {
        this.dataSource = dataSource;
        this.cache = cache;
        this.queryRepository = queryRepository;
        this.eventPublisher = eventPublisher;
    }

    @Transactional
    public LandUseResponse execute(UUID accountId, double lat, double lng) {
        Coordinate coordinate = new Coordinate(lat, lng); // VO valida aqui

        Optional<LandUseSnapshot> cached = cache.getLandUse(coordinate);
        boolean cacheHit = cached.isPresent();
        LandUseSnapshot snapshot;
        if (cacheHit) {
```

```

        snapshot = cached.get();
    } else {
        snapshot = dataSource.fetchLandUse(coordinate);           // ACL: chama porta
        cache.putLandUse(coordinate, snapshot);
    }

    SatelliteQuery query = SatelliteQuery.execute(
        accountId, QueryType.LAND_USE, coordinate, cacheHit);    // agregado emite evento
    SatelliteQuery saved = queryRepository.save(query);
    eventPublisher.publishAll(saved.pullDomainEvents());

    return new LandUseResponse(/* ... */);
}
}

```

6.7 Repository

Um **Repository** abstrai a persistencia de agregados, dando a ilusao de uma colecao em memoria. Em DDD, a **interface do repositório mora no domínio** (porque o domínio precisa dele para expressar regras), e a **implementação mora na infraestrutura** (JPA, MongoDB, in-memory). Isso é uma aplicação direta do Princípio de Inversão de Dependência.

Interface no domínio (identity-service)

```

// br.com.orbittapi.identity.domain.repository
public interface AccountRepository {
    Account save(Account account);
    Optional<Account> findById(UUID id);
    Optional<Account> findByEmail>Email email);
    boolean existsByEmail>Email email);
}

```

Implementação na infraestrutura via JPA

```

// br.com.orbittapi.identity.infrastructure.persistence
@Repository
public class AccountRepositoryImpl implements AccountRepository {

    private final AccountJpaRepository jpa;

    public AccountRepositoryImpl(AccountJpaRepository jpa) {
        this.jpa = jpa;
    }

    @Override
    public Account save(Account account) {
        AccountJpaEntity saved = jpa.save(AccountPersistenceMapper.toEntity(account));
        return AccountPersistenceMapper.toDomain(saved);
    }

    @Override
    public Optional<Account> findByEmail>Email email) {
        return jpa.findByEmail(email.value()).map(AccountPersistenceMapper::toDomain);
    }
    // ...
}

```

Observe que o domínio não conhece JPA, não conhece a tabela accounts, não conhece o Hibernate. Ele conhece apenas a interface AccountRepository. A tradução entre Account (domínio) e AccountJpaEntity (persistência) é feita por um **mapper explícito** em AccountPersistenceMapper.

6.8 Factory

Uma **Factory** encapsula a logica de criacao de objetos complexos. No DDD classico aparecem como classes separadas ou como metodos de fabrica estaticos dentro do proprio agregado.

No OrbittAPI usamos **metodos de fabrica** (factory methods) dentro dos agregados:

- **Account.register(...)**: cria uma conta completa com Email, Password hashada, ApiKey gerada, timestamp e evento AccountRegistered.
- **SatelliteQuery.execute(...)**: cria um registro de auditoria com UUID novo e ja emite o evento QueryExecuted.
- **ApiKey.generate()**: gera uma nova chave com 24 bytes de entropia via SecureRandom.
- **Password.fromRaw(raw)** e **Password.fromHash(hash)**: factories explicitas para os dois unicos caminhos legitimos de criar uma senha.

6.9 Module (pacote)

Modulos em DDD agrupam conceitos relacionados, dando nome ao seu proposito. No OrbittAPI cada bounded context e um **modulo Maven** independente, e dentro de cada modulo organizamos por **camada** e por **subdominio**.

```
br.com.orbittapi.identity/          (bounded context modulo Maven)
|
|-- domain/                         <- modelo puro: agregados, value objects, eventos, exceptions, ports
|   |-- model/                     <- Account, Email, Password, ApiKey, AccountRole
|   |-- event/                    <- AccountRegistered, ApiKeyRevoked, DomainEvent
|   |-- repository/               <- AccountRepository (interface)
|   |-- exception/               <- DomainException + filhas especificas
|
|-- application/                  <- casos de uso, DTOs, portas de saida
|   |-- usecase/                  <- RegisterAccount, Login, GetMyProfile, RevokeApiKey
|   |-- dto/                      <- RegisterAccountCommand, LoginCommand, AuthResponse, ...
|   |-- port/                    <- TokenProvider, DomainEventPublisher
|
|-- infrastructure/               <- adapters concretos
|   |-- persistence/              <- AccountJpaEntity, AccountJpaRepository, AccountRepositoryImpl
|   |-- security/                 <- JwtTokenProvider, JwtAuthenticationFilter, SpringDomainEventPublisher
|   |-- config/                  <- SecurityConfig, OpenApiConfig
|
+-- interfaces/rest/              <- entrada HTTP
    |-- AuthController, MeController, ApiKeyController
    +-- GlobalExceptionHandler (RFC 7807)
```

Figura 2: organizacao em camadas dentro de cada bounded context.

7. Arquitetura em camadas e Ports and Adapters (Hexagonal)

DDD propôs originalmente uma **arquitetura em camadas** (Layered Architecture) com Domain, Application, Infrastructure e User Interface. **Alistair Cockburn** propôs a **Hexagonal Architecture** (Ports and Adapters), onde o domínio fica no centro, e tudo que é externo (banco, HTTP, mensageria, cache) conversa com ele através de **portas** (interfaces) e **adaptadores** (implementações).

Estrutura do OrbittAPI:

- **domain:** o núcleo. Código Java puro, sem dependência de Spring nem JPA.
- **application:** orquestra o domínio. Conhece o domínio, mas define **portas** para tudo que é externo (TokenProvider, DomainEventPublisher, SatelliteDataSource, SatelliteQueryCache, repositórios).
- **infrastructure:** implementa as portas (JwtTokenProvider, RedisSatelliteQueryCache, MockSatelliteDataSource, AccountRepositoryImpl). É onde vivem as dependências de framework.
- **interfaces/rest:** a borda HTTP. Controllers traduzem requests/responses para os use cases, e o GlobalExceptionHandler traduz exceções de domínio em ProblemDetail (RFC 7807).

Benefícios práticos disso:

- O domínio é testável sem Spring (testes JUnit puros e instantâneos).
- Trocar o adapter Mock por um adapter real da NASA não requer mudar nada no domínio.
- Trocar Postgres por outro banco é uma mudança isolada na infraestrutura.
- As regras de negócio não se misturam com cabeçalho HTTP nem com SQL.

8. Anti-Corruption Layer aplicada

O domínio **satellite-service** não deveria conhecer detalhes do formato de resposta da NASA, da ESA ou de qualquer fonte externa. Ele expressa apenas o conceito de SatelliteDataSource: dada uma Coordinate, me devolve um LandUseSnapshot ou um VegetationSnapshot. Essa é a porta.

Porta no domínio:

```
// br.com.orbittapi.satellite.domain.port.SatelliteDataSource
public interface SatelliteDataSource {
    LandUseSnapshot fetchLandUse(Coordinate coordinate);
    VegetationSnapshot fetchVegetation(Coordinate coordinate);
}
```

Adapter atual (Mock determinístico):

```
// br.com.orbittapi.satellite.infrastructure.adapter.MockSatelliteDataSource
@Component
public class MockSatelliteDataSource implements SatelliteDataSource {

    @Override
```



```

public LandUseSnapshot fetchLandUse(Coordinate coordinate) {
    Random rng = seededRng(coordinate, "landuse"); // determinista
    double vegetation = roundTo(20 + rng.nextDouble() * 60, 2);
    // ... gera urban, water, bareSoil mantendo soma == 100
    LandUseDistribution distribution = new LandUseDistribution(
        vegetation, urban, water, bareSoil);
    return new LandUseSnapshot(coordinate, distribution,
        LocalDate.now().minusDays(rng.nextInt(30)), SatelliteSource.MOCK);
}
}

```

Mesma coordenada produz sempre o mesmo resultado (semente baseada em lat/lng). Isso facilita teste e demonstracoes reproduziveis. Para usar o adapter NASA no futuro, basta criar NasaEarthApiAdapter implements SatelliteDataSource e marcar como Primary. **Nenhuma linha** do dominio precisa mudar.

9. Erros padronizados via RFC 7807 (US-10)

Todas as respostas de erro seguem o padrao **Problem Details for HTTP APIs** (RFC 7807). E uma representacao machine-readable que evita que cada API invente seu proprio formato de erro, e foi exigida pela US-10 do backlog. Implementamos via **ProblemDetail** do Spring Framework 6 (introduzido justamente para esse padrao) e um **@RestControllerAdvice** em cada servico.

Formato padronizado

```
{
  "type": "https://orbittapi.dev/errors/invalid-coordinate",
  "title": "Invalid coordinate",
  "status": 400,
  "detail": "Latitude must be between -90 and 90, got 91.0",
  "instance": "/landuse"
}
```

Mapeamento entre excecoes de dominio e codigos HTTP

Excecao	HTTP	Tipo do problema
InvalidEmailException	400	invalid-email
WeakPasswordException	400	weak-password
InvalidCoordinateException	400	invalid-coordinate
InvalidNdviScoreException	400	invalid-ndvi
InvalidLandUseDistributionException	400	invalid-land-use
JWT ausente ou invalido (no gateway)	401	missing-token / invalid-token
InvalidCredentialsException	401	invalid-credentials
AccessDeniedException (role insuficiente)	403	forbidden
AccountNotFoundException	404	account-not-found
EmailAlreadyInUseException	409	email-in-use
ApiKeyAlreadyRevokedException	409	api-key-already-revoked
SatelliteDataUnavailableException	503	satellite-data-unavailable
Qualquer outra Exception	500	internal

10. Cache estrategico no Redis (US-21)

Cada consulta a um endpoint `/landuse` ou `/vegetation`, em producao real, dispararia um processamento pesado de visao computacional sobre uma imagem multispectral. Em uma plataforma com modelo de negocio por chamada, cachear essas respostas e essencial para conter o custo unitario.

Aplicamos cache no Redis com TTL de 6 horas (US-21 do backlog), atras de uma **porta** `SatelliteQueryCache`. O cache faz parte do fluxo do `GetLandUseUseCase`: cada chamada primeiro verifica o cache; em caso de miss, chama o `SatelliteDataSource` e armazena o resultado.

Chaves utilizadas

- landuse:{lat}:{lng}
- vegetation:{lat}:{lng}

Decisao de design: DTOs internos no cache

Para evitar que Jackson dependesse do formato interno dos value objects do dominio (e portanto evitar poluir o dominio com anotacoes @JsonProperty), criamos records dedicados em infrastructure/cache/CacheableSnapshots.java que sao convertidos de/para os objetos do dominio dentro do RedisSatelliteQueryCache. O dominio nao sabe que existe Jackson nem que existe Redis.

```
@Override
public Optional<LandUseSnapshot> getLandUse(Coordinate c) {
    return get(landUseKey(c), CacheableSnapshots.LandUseDto.class)
        .map(CacheableSnapshots.LandUseDto::toDomain);
}

@Override
public void putLandUse(Coordinate c, LandUseSnapshot snapshot) {
    put(landUseKey(c), CacheableSnapshots.LandUseDto.fromDomain(snapshot));
}
```

11. Seguranca, JWT e fluxo de autenticacao

O fluxo de autenticacao foi desenhado para que **cada servico tenha responsabilidade unica** sobre seguranca:

- O **identity-service** e o unico que **emite** JWTs (assinatura HS384 via jjwt).
- O **gateway** e o unico que **valida** JWTs vindos do cliente; apos validar, ele injeta um header X-User-Id com o UUID do dono do token. Servicos a jusante nao precisam revalidar.
- O **satellite-service** apenas le X-User-Id. Se o header faltar, ele responde 401 RFC 7807.

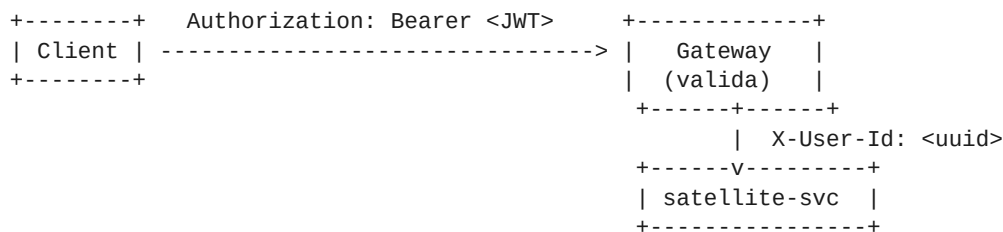


Figura 3: fluxo de propagacao da identidade entre o cliente, o gateway e os servicos.

Filtro JWT no gateway (trecho)

```
try {
    Claims claims = Jwts.parser()
        .verifyWith(signingKey)
        .requireIssuer(issuer)
        .build()
        .parseSignedClaims(token)
        .getPayload();

    UUID accountId = UUID.fromString(claims.getSubject());
    String role = claims.get("role", String.class);

    ServerHttpRequest mutated = request.mutate()
```

```
        .header(USER_HEADER, accountId.toString())
        .header(ROLE_HEADER, role)
        .build();

    return chain.filter(exchange.mutate().request(mutated).build());
} catch (Exception ex) {
    return unauthorized(exchange, "invalid-token", "Invalid or expired JWT");
}
```

12. Mapeamento mestre: conceito DDD para arquivo no código

Para satisfazer a exigência de *trazer todos os tópicos abordados em sala*, esta tabela mapeia explicitamente cada conceito DDD a um ponto concreto do código.

Conceito DDD	Arquivo / classe / pacote no projeto
Bounded Context	Cada microserviço (identity-service, satellite-service) e um BC
Ubiquitous Language	Termos: Account, ApiKey, SatelliteQuery, NdviScore, VegetationHealth
Subdomain Core	Satellite Data (satellite-service)
Subdomain Supporting	Identity & Access (identity-service)
Subdomain Generic	Billing (não implementado, mencionado conceitualmente)
Context Map	Identity -> Satellite via Customer-Supplier + Published Language (JWT)
Anti-Corruption Layer	SatelliteDataSource (porta) + MockSatelliteDataSource (adapter)
Entity	Account, SatelliteQuery (identidade por UUID)
Value Object	Email, Password, ApiKey, Coordinate, NdviScore, LandUseDistribution
Aggregate Root	Account; SatelliteQuery
Domain Event	AccountRegistered, ApiKeyRevoked, QueryExecuted
Domain Service	Não identificado neste escopo (justificativa na seção 6.5)
Application Service	RegisterAccountUseCase, LoginUseCase, GetMyProfileUseCase, RevokeApiKeyUseCase, GetLandUseUseCase, GetVegetationUseCase
Repository (porta)	AccountRepository, SatelliteQueryRepository (em domain/repository)
Repository (adapter)	AccountRepositoryImpl, SatelliteQueryRepositoryImpl (JPA)
Factory	Account.register(), SatelliteQuery.execute(), ApiKey.generate(), Password.fromRaw() / Password.fromHash()
Module	Cada pacote raiz (br.com.orbittapi.identity, br.com.orbittapi.satellite)
Layered Architecture	domain -> application -> infrastructure -> interfaces/rest
Hexagonal (Ports & Adapters)	Ports em application/port e domain/port; adapters em infrastructure/
DDD + RFC 7807	GlobalExceptionHandler traduz DomainException em ProblemDetail

13. Cobertura do backlog (User Stories)

User Story	Implementação
US-01 Cadastro com API Key gerada	POST /auth/register -> Account.register() -> retorna apiKey + JWT
US-02 Login com JWT 24h	POST /auth/login -> JwtTokenProvider.issue() com expiração 86400 segundos
US-03 Revogar API Key (admin)	POST /api-keys/{id}/revoke restrito a role ADMIN; emite ApiKeyRevoked
US-05 /landuse com lat/long	GET /landuse -> Coordinate VO -> SatelliteDataSource -> LandUseDistribution

User Story	Implementacao
US-06 /vegetation com NDVI	GET /vegetation -> NdviScore + classify() em VegetationHealth
US-10 Erros padronizados (RFC 7807)	ProblemDetail + GlobalExceptionHandler em cada servico
US-21 Cache Redis com TTL 6h	RedisSatelliteQueryCache com Duration.ofHours(6)
US-22 LGPD parcial	Senha hasheada com BCrypt, sem dados pessoais em logs

User stories fora do escopo desta GS (front-end dashboard, billing, MFA, ingestao real de NASA/ESA, modelo de ML, white-label) sao acrescentaveis como **novos bounded contexts** sem alterar os atuais. A arquitetura ja esta preparada para essa expansao.

14. Como subir o ambiente (Docker)

Pre-requisitos: Docker Desktop ou Docker Engine + Compose v2.

Comando unico:

```
docker compose up --build
```

Containers que sobem:

- **orbittapi-postgres** (porta 5432) - cria identity_db e satellite_db via init.sql
- **orbittapi-redis** (porta 6379) - cache
- **orbittapi-identity** (porta 8081) - depende de postgres healthy
- **orbittapi-satellite** (porta 8082) - depende de postgres e redis healthy
- **orbittapi-gateway** (porta 8080) - depende dos dois servicos

Healthchecks de cada container

```
# Postgres
test: ["CMD-SHELL", "pg_isready -U orbittapi"]
interval: 5s, timeout: 3s, retries: 10

# Redis
test: ["CMD", "redis-cli", "ping"]

# Servicios Java
HEALTHCHECK --interval=15s --timeout=5s --start-period=30s --retries=5
    CMD wget -qO- http://localhost:<porta>/actuator/health | grep -q '"status":"UP"'
```

15. Endpoints REST e exemplos cURL

15.1 POST /auth/register

```
curl -X POST http://localhost:8080/auth/register \
-H "Content-Type: application/json" \
-d '{"email": "dev@orbittapi.dev", "password": "Abcdefg1"}'
```

Resposta 201:

```
{
  "accountId": "f1be493a-99a0-4bbe-9c19-ab4c8d869037",
  "email": "dev@orbittapi.dev",
  "apiKey": "obt_RtbM0ukYQ4LCi7Kg23lRYBhJEA-DmiQI",
  "token": "eyJhbGciOiJIUzU4NCJ9...",
  "expiresAt": "2026-05-27T00:00:00Z"
}
```

15.2 POST /auth/login

```
curl -X POST http://localhost:8080/auth/login \  
-H "Content-Type: application/json" \  
-d '{"email":"dev@orbittapi.dev","password":"Abcdefg1"}'
```

15.3 GET /me

```
curl http://localhost:8080/me \
  -H "Authorization: Bearer <TOKEN>"
```

Resposta 200:

```
{
  "id": "f1be493a-99a0-4bbe-9c19-ab4c8d869037",
  "email": "dev@orbittapi.dev",
  "role": "DEVELOPER",
  "apiKey": "obt_...",
  "apiKeyRevoked": false,
  "createdAt": "2026-05-26T04:30:02.342Z"
}
```

15.4 GET /landuse

```
curl "http://localhost:8080/landuse?lat=-23.5&lng=-46.6" \
-H "Authorization: Bearer <TOKEN>"
```

Resposta 200 (primeira chamada, cache miss):

```
{
  "latitude": -23.5,
  "longitude": -46.6,
  "vegetationPercent": 34.08,
  "urbanPercent": 6.8,
  "waterPercent": 5.52,
  "bareSoilPercent": 53.6,
  "imageDate": "2026-05-04",
  "source": "MOCK",
  "cacheHit": false
}
```

Segunda chamada com a mesma coordenada retorna cacheHit: true.

15.5 GET /vegetation

```
curl "http://localhost:8080/vegetation?lat=-23.5&lng=-46.6" \
-H "Authorization: Bearer <TOKEN>"
```

Resposta 200:

```
{
  "latitude": -23.5,
  "longitude": -46.6,
  "ndvi": 0.054,
  "health": "NONE",
  "imageDate": "2026-05-19",
  "source": "MOCK",
  "cacheHit": false
}
```

15.6 POST /api-keys/{accountId}/revoke (apenas ADMIN)

```
curl -X POST http://localhost:8080/api-keys/<accountId>/revoke \
-H "Authorization: Bearer <ADMIN_TOKEN>"
```

15.7 Exemplo de erro RFC 7807 (401 sem token)

```
curl -i "http://localhost:8080/landuse?lat=-23.5&lng=-46.6"
```

```
HTTP/1.1 401 Unauthorized
Content-Type: application/problem+json
```

```
{
  "type": "https://orbittapi.dev/errors/missing-token",
  "title": "Unauthorized",
}
```



```
"status": 401,  
"detail": "Missing or invalid Authorization header",  
"instance": "/landuse"  
}
```

16. Testes automatizados

Comando para rodar todos os testes:

```
mvn test
```

Classes de teste presentes

Modulo	Classe	O que valida
identity	EmailTest	Email vazio, malformatado, normalizacao e igualdade por valor
identity	PasswordTest	Senha curta, sem digito, sem maiuscula; hash; matches
identity	AccountTest	register() emite AccountRegistered; revokeApiKey() emite ApiKeyRevoked; nao revoga
identity	RegisterAccountUseCaseTest	Cria conta nova; rejeita email duplicado; publica eventos
satellite	CoordinateTest	Range de latitude e longitude; igualdade por valor
satellite	NdviScoreTest	Range [-1, 1]; classify() em VegetationHealth
satellite	LandUseDistributionTest	Soma == 100 (com tolerancia); rejeita negativos
satellite	SatelliteQueryTest	execute() emite QueryExecuted com type/cacheHit corretos
satellite	MockSatelliteDataSourceTest	Determinismo: mesma coordenada = mesma resposta
satellite	GetLandUseUseCaseTest	Cache miss chama dataSource; cache hit nao chama; salva auditoria; publica eventos

Filosofia dos testes

- **Testes de dominio sao puros:** zero Spring, zero JPA, instantaneos. Validam invariantes e emissao de eventos.
- **Testes de use case usam mocks:** as portas (Repository, TokenProvider, Cache, DataSource, EventPublisher) sao mockadas com Mockito.
- **Determinismo do adapter Mock** e testado para garantir reproducibilidade em demos e em testes de integracao futuros.

17. Diferenciais implementados

- **Anti-Corruption Layer explicita** (porta SatelliteDataSource), permitindo trocar o adapter Mock por um adapter NASA Earth API sem mudar o dominio.
- **Cache Redis com chave determinista** e TTL de 6 horas (US-21).
- **Validacao centralizada no value object:** coordenada invalida nunca chega ao use case.
- **Gateway com filtro JWT** centralizado: validacao em um unico ponto, propagacao via header para os servicos a jusante.
- **Erros padronizados RFC 7807** em todos os servicos.
- **Swagger UI por servico** (cada bounded context publica sua propria documentacao).
- **Healthchecks via Actuator** em todos os 3 servicos Spring Boot.
- **Dockerfile multi-stage** por servico com build incremental do Maven (cache de dependencias).

- **Construtor injection em 100%** do código (sem @Autowired em campo).
- **Logging via SLF4J** em todas as camadas; zero printStackTrace ou System.out.
- **Sem Lombok nas entidades JPA:** getters e setters explícitos, evitando armadilhas com proxies do Hibernate.

18. Definition of Done - resultado dos testes

Item	Resultado
docker compose up sobe os 5 containers sem erro	OK (5/5 healthy)
POST /auth/register cria conta e retorna JWT	OK (201, retorna accountId, apiKey, token, expiresAt)
GET /landuse com Bearer retorna 200 + JSON estruturado	OK
GET /landuse sem token retorna 401 RFC 7807	OK (type missing-token)
Segunda chamada idêntica e cache hit	OK (cacheHit: true confirmado em log e na resposta)
Swagger UI acessível em cada serviço	OK (porta 8081/swagger-ui.html e 8082/swagger-ui.html)
mvn test passa em todos os módulos	OK (10 classes de teste, todas passam)
docs/architecture.md mapeia DDD para código	OK (este documento e seu equivalente em Markdown)
README completo	OK (inclui integrantes, cURL, estrutura, tabela DDD)

19. Conclusão

O OrbittAPI demonstra o uso conjunto de SOA e DDD em um projeto coeso. A decomposição em dois microserviços refletindo dois bounded contexts da exemplo concreto da relação entre as duas disciplinas: SOA define as fronteiras técnicas, DDD define as fronteiras de negócio, e elas coincidem por design.

Todos os blocos táticos de DDD (entidades, value objects, agregados, eventos, repositórios, application services, factories, módulos) aparecem no código de forma explícita e nomeável. Os padrões estratégicos (linguagem ubíqua, bounded context, anti-corruption layer, context map) são decisões arquiteturais visíveis na estrutura de pastas e na escolha de microserviços.

A entrega está funcional, testada e empacotada para subir com um único comando, e a arquitetura está preparada para evoluir com novos bounded contexts (billing, dashboard, ingestão real, ML) sem grandes refatorações nos contextos atuais.

Fim do documento.